SQL, Plans and Instruments from Zero

The hands-on half of the midterm: the queries you must write, how to read an EXPLAIN plan line by line, and every PostgreSQL instrument the mocks use, each tied to the question that needs it

Advanced Data Management

2026-10-31

How to use this

Three parts, in the order you need them. **Part I** is the SQL the mocks make you write (Mock 1 3.3 and 3.7, the lab queries), built clause by clause; nothing is assumed beyond `SELECT ... FROM ... WHERE`. **Part II** is plan reading: what `EXPLAIN`, `EXPLAIN ANALYZE` and `EXPLAIN (ANALYZE, BUFFERS)` each print, the anatomy of a node line, the catalogue of nodes, a five-move method, the three plans of the mocks worked in full, then five fresh plans to practise on with solutions underneath (cover them). **Part III** is the instruments: for each PostgreSQL view, extension or setting the mocks rely on, the exact command, a sample of its output, how to read it, and which mock question it answers. The last page maps every hands-on mock question to the sections here. Everything runs on `adm_lab`: `customers` 10,000 rows / 73 pages, `orders` 500,000 rows / 3,185 pages / 157 rows per page, `amount` uniform 5..105, 200 products, 365 order dates; Mock 1's `shopstream` numbers (`events` 50M rows / 1M pages / 50 per page) are used where its questions are.

Contents

I The SQL you must be able to write **3**

1 Read this first: how a query is evaluated **3**

2 The report query (Mock 1, 3.7a; Lab 1; drill D3) **3**

3 Generating data: `generate_series` (Mock 1, 3.7b; Labs; drill D1–D2) **4**

4 Creating indexes (Mock 1, 3.3; Mock v2 A3; Lab 3; drill D5) 6

5 psql survival kit 7

II Reading a plan, from zero 7

6 EXPLAIN, EXPLAIN ANALYZE, EXPLAIN (ANALYZE, BUFFERS): what each one does 8

7 Anatomy of one node line 10

8 The tree: who feeds whom 10

9 The nodes you will meet 11

10 The five-move method 12

11 Worked plan 1: Lab 4 part 1 12

12 Worked plan 2: Mock v2 Part C, the December report (26 pts) 13

13 Worked plan 3: Mock 1, 3.1, the 13-minute query 14

14 Practice plans (cover the solutions) 15

III The PostgreSQL instruments, one per mock question 18

15 Page counts: `pg_class`, `pg_relation_size` 18

16 Looking inside a page: `pageinspect`, `ctid` 19

17 What the planner believes about a column: `pg_stats` 19

18 Fresh or stale: `ANALYZE`, `pg_stat_user_tables` 21

19 Correlated columns: `CREATE STATISTICS` 21

20 Steering the planner: `SET enable_*`, `work_mem`, `SET LOCAL`, `RESET` 22

21 The buffer pool: pg_buffercache and the Buffers line 22

22 Which indexes are used: pg_stat_user_indexes, pg_indexes 23

23 One column out: COPY to Parquet and DuckDB 24

24 The map: mock question to what you type 25

Part I

The SQL you must be able to write

1 Read this first: how a query is evaluated

Concept: The six clauses, in the order PostgreSQL runs them

Runs	Clause	Question it answers	Produces
1	`FROM a JOIN b ON ...`	which tables, paired how	one wide row per matching pair
2	`WHERE`	which *rows* to keep	fewer rows
3	`GROUP BY`	collapse rows with equal keys	one row per group
4	`HAVING`	which *groups* to keep	fewer groups
5	`SELECT`	what to output; aliases are born here	columns and aggregates
6	`ORDER BY` / `LIMIT`	final order, how many	the result

You *write* SELECT first, but it *runs* fifth. Hence the three rules: an alias from SELECT can be used in ORDER BY but not in WHERE; an aggregate (`count`, `sum`) may not appear in WHERE, it goes in HAVING; every column in SELECT must be in GROUP BY or inside an aggregate.

2 The report query (Mock 1, 3.7a; Lab 1; drill D3)

Asked: "the number of purchase events per country in the last 30 days, biggest first". Build it clause by clause.

1. **Which tables?** `country` is in `users`, the events are in `events`: two tables, so a join. They share `user_id`: `FROM events e JOIN users u ON u.user_id = e.user_id`. The letters `e`, `u` are *aliases*: short names you must then use everywhere.
2. **Which rows?** Only purchases, only recent: `WHERE e.event_type = 'purchase' AND e.ts >= date '2026-10-12' - interval '30 days'`. Strings in single quotes. `date '...'` and `interval '...'` are typed literals; date minus interval is a timestamp. With today's date unknown

write `now() - interval '30 days'`.

3. **Per what?** Per country: `GROUP BY u.country`.
4. **Count what?** `count(*)` counts rows per group. Give it a name: `AS purchases`.
5. **Order:** biggest first: `ORDER BY purchases DESC` (the alias is legal here, clause 6 runs after 5).

```
SELECT   u.country, count(*) AS purchases
FROM     events e JOIN users u ON u.user_id = e.user_id
WHERE    e.event_type = 'purchase'
  AND    e.ts >= date '2026-10-12' - interval '30 days'
GROUP BY u.country
ORDER BY purchases DESC;
```

Variants the exam could ask (same skeleton, one clause changes):

```
-- revenue per country, December only, biggest first     (Mock v2, Part C
    query)
SELECT c.country, sum(o.amount) AS revenue
FROM   orders o JOIN customers c ON c.id = o.cust_id
WHERE  o.order_date >= '2025-12-01'
GROUP BY c.country ORDER BY revenue DESC;
-- only countries with more than 100 purchases: a condition on a GROUP goes
    in HAVING
... GROUP BY u.country HAVING count(*) > 100 ORDER BY purchases DESC;
-- per day instead of per country: truncate the timestamp
SELECT date_trunc('day', e.ts) AS day, count(*) FROM events e GROUP BY day
    ORDER BY day;
-- two counts in one pass: FILTER
SELECT count(*) FILTER (WHERE event_type = 'purchase') AS purchases,
       count(*) FILTER (WHERE event_type = 'view')     AS views FROM events;
-- the fraction of a table that satisfies a predicate (real selectivity, Lab
    5.3, drill D4)
SELECT count(*) FILTER (WHERE country = 'MA')::float / count(*) FROM
    customers;   -- ~0.20
```

Warning / Gotcha: The three ways to lose the 4 points

`WHERE purchases > 100` (alias does not exist yet) or `WHERE count(*) > 100` (aggregate in WHERE): both are errors, use HAVING. `SELECT u.country, u.city, count(*) ... GROUP BY u.country`: `city` is neither grouped nor aggregated, error. Forgetting the `ON` condition: a cross product of 50M × 2M rows.

3 Generating data: `generate_series` (Mock 1, 3.7b; Labs; drill D1–D2)

Mechanism: `generate_series` is a row factory

`SELECT g FROM generate_series(1, 5) AS g;` returns five rows, 1 to 5. Put a `generate_series` in FROM and every expression in the SELECT

list is evaluated *once per row*: that is how one statement inserts a million rows. The building blocks:

```
random()                                              -- a double in [0,1),
     new for every row
1 + floor(random() * 10000)::int                      -- a random integer
    1..10000 (::int is the cast)
round((random() * 100 + 5)::numeric, 2)               -- a random amount
    5.00..105.00
date '2025-01-01' + floor(random() * 365)::int        -- a random day of
    2025 (date + integer days)
now() - random() * interval '365 days'                -- a random timestamp
     in the last year
(ARRAY['web','ios','android'])[1 + floor(random()*3)::int]   -- a random
    element of a list
CASE WHEN r < 0.9 THEN 'view' WHEN r < 0.995 THEN 'cart' ELSE 'purchase'
    END  -- skewed category
```

```
-- the two lab tables, from nothing (drill D1)
CREATE TABLE customers (id serial PRIMARY KEY, name text, country text, city
    text);
CREATE TABLE orders (id serial PRIMARY KEY, cust_id int REFERENCES customers(
    id),
                     product_id int, amount numeric(10,2), order_date date);
INSERT INTO customers (name, country)
SELECT 'cust_' || g, (ARRAY['MA','FR','ES','DE','UK'])[1 + floor(random() *
    5)::int]
FROM generate_series(1, 10000) g;
INSERT INTO orders (cust_id, product_id, amount, order_date)
SELECT 1 + floor(random() * 10000)::int, 1 + floor(random() * 200)::int,
       round((random() * 100 + 5)::numeric, 2), date '2025-01-01' + floor(
    random() * 365)::int
FROM generate_series(1, 500000);
ANALYZE;                                    -- always, after a bulk load:
    statistics (Part III)
```

The skewed insert (Mock 1, 3.7b): 90% view, 9.5% cart, 0.5% purchase. Draw *one* random number per row, call it `r`, and cut it at 0.90 and 0.995:

```
INSERT INTO events (user_id, event_type, ts, product_id, payload, device)
SELECT 1 + floor(random() * 2000000)::int,
       CASE WHEN r < 0.90 THEN 'view' WHEN r < 0.995 THEN 'cart' ELSE '
    purchase' END,
       timestamp '2026-10-12' - random() * interval '365 days',
       1 + floor(random() * 50000)::int,
       '{}'::jsonb,
       (ARRAY['web','ios','android'])[1 + floor(random() * 3)::int]
FROM (SELECT random() AS r FROM generate_series(1, 1000000)) g;
ANALYZE events;
-- check:  SELECT event_type, count(*) FROM events GROUP BY 1;   -- ~900,000
    / 95,000 / 5,000
```

Warning / Gotcha: One draw, several cuts

`CASE WHEN random() < 0.9 THEN 'view' WHEN random() < 0.995 THEN 'cart' ...` calls `random()` twice with two different numbers: a row that failed the first test (10% of rows) passes the second with probability 0.995, so you get 0.9 / 0.0995 / 0.0005, not 0.9 / 0.095 / 0.005. The subquery that names `r` is the whole trick. `GROUP BY 1` means "by the first output column".

4 Creating indexes (Mock 1, 3.3; Mock v2 A3; Lab 3; drill D5)

Interface / API: Every `CREATE INDEX` form the exam can ask

```
CREATE INDEX ON orders (cust_id);                          -- B-tree (
    default): =, <, BETWEEN, ORDER BY
CREATE INDEX idx_orders_date ON orders (order_date);       -- same, with a
     name you choose
CREATE UNIQUE INDEX ON users (email);                      -- unique
    lookup; also enforces uniqueness
CREATE INDEX ON orders (cust_id, order_date);              -- composite:
    leading column first (A3)
CREATE INDEX ON users (lower(email));                      -- expression
    index: WHERE lower(email) = $1
CREATE INDEX ON events (user_id) WHERE event_type = 'purchase';  --
    partial: 0.5% of the rows
CREATE INDEX ON orders (cust_id) INCLUDE (amount);         -- covering:
    index-only scan for sum(amount)
CREATE INDEX ON events USING brin (ts);                    -- BRIN: tiny,
    for a table appended in ts order
CREATE INDEX ON users USING hash (email);                  -- hash:
    equality only, no ranges
DROP INDEX idx_orders_date;
```

Which one, per pattern (the answer sheet of Mock 1, 3.3):

Pattern	Index	One-line reason
(a) `WHERE email = $1`	`CREATE UNIQUE INDEX ON users(email)`	one row of 2M: B-tree, 4 page reads; unique enforces the rule
(b) `WHERE ts BETWEEN $1 AND $2`	`CREATE INDEX ON events(ts)`, or `USING brin` if appended in time order	B-tree keeps order, so a range walks the leaves; BRIN if correlation ≈ 1
(c) `WHERE event_type = 'purchase'`	`CREATE INDEX ON events(user_id) WHERE event_type = 'purchase'`	0.5% of rows: a partial index 200× smaller; keyed on `user_id` it also serves the join
(d) `WHERE event_type = 'view'`	**none**	90% of 50M rows on all 1M pages; threshold $1/(4 \cdot 50) = 0.5\%$; the planner scans
(e) `WHERE lower(email) = $1`	`CREATE INDEX ON users(lower(email))`	an index on `email` is sorted by `email`, not by `lower(email)`
(f) `JOIN ... ON e.user_id = u.user_id`	`CREATE INDEX ON events(user_id)`	the foreign-key side: index nested loop for small outers

After creating one, prove it is used: `EXPLAIN SELECT ... WHERE email = 'x';` must show `Index Scan using ...` (or `Bitmap Index Scan`); if it still shows `Seq Scan`, the predicate keeps too many rows, or the statistics are stale (`ANALYZE`), or a function hides the column.

5 psql survival kit

```
\timing on          -- print elapsed ms after every statement (Lab 4 timings)
\d orders           -- columns, types, indexes of a table      \di   -- list
    indexes
\x                  -- vertical output: use it before SELECT * FROM pg_stats
    (wide rows)
\dt                 -- list tables                                \q   -- quit
BEGIN; EXPLAIN ANALYZE DELETE FROM orders WHERE ...; ROLLBACK;  -- ANALYZE
    really runs DML
```

Part II

Reading a plan, from zero

6 EXPLAIN, EXPLAIN ANALYZE, EXPLAIN (ANALYZE, BUFFERS): what each one does

Concept: One idea

`EXPLAIN` shows you the **plan**: the tree of operators the planner chose for a query, with its *predictions*. Nothing else in PostgreSQL shows you that. The options only decide *how much* is printed next to the plan, and whether the query is actually run to get real measurements.

The same query three ways. `SELECT * FROM orders WHERE amount > 90;` (`amount` uniform 5..105, so 15% of the rows):

```
EXPLAIN SELECT * FROM orders WHERE amount > 90;
Seq Scan on orders  (cost=0.00..9435.00 rows=75000 width=24)
  Filter: (amount > '90'::numeric)
                                        -- a prediction. Did not run. No times, no
     actual rows.
EXPLAIN ANALYZE SELECT * FROM orders WHERE amount > 90;
Seq Scan on orders  (cost=0.00..9435.00 rows=75000 width=24) (actual time=0.015..52.3
      rows=74812 loops=1)
  Filter: (amount > '90'::numeric)
  Rows Removed by Filter: 425188
Planning Time: 0.09 ms
Execution Time: 55.1 ms                 -- ran it: a second pair of parentheses per
     node, with measurements
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE amount > 90;
Seq Scan on orders  (cost=0.00..9435.00 rows=75000 width=24) (actual time=0.015..52.3
      rows=74812 loops=1)
  Filter: (amount > '90'::numeric)
  Rows Removed by Filter: 425188
  Buffers: shared hit=3185              -- plus the pages: 3,185 = all of orders, all
      from the pool
Planning Time: 0.09 ms
Execution Time: 55.1 ms
```

Read the first pair of parentheses as "*the planner thinks*" and the second as "*what happened*". The exam's diagnosis questions are always about the two `rows=` disagreeing; the storage and buffer questions are always about the `Buffers` line. **Which form, when.**

Form	Runs the query?	What you get	Use it when
`EXPLAIN q;`	**no**	the plan and the planner's *estimates* only: `cost=`, `rows=`, `width=`	you want to know what the planner *would* do (is my index used? which join?) without paying for the query; safe on a 13-minute query
`EXPLAIN ANALYZE q;`	**yes**	the same, plus for every node the *measured* `actual time=a..b`, `rows=`, `loops=`, plus `Sort Method`, `Batches`, `Rows Removed by Filter`, `Heap Blocks`, and `Execution Time`	you want estimate *against* actual: diagnosing a bad estimate, timing three join algorithms (Lab 4), seeing whether a sort spilled
`EXPLAIN (ANALYZE, BUFFERS) q;`	**yes**	all of the above, plus per node the `Buffers:` line: `shared hit=` (pages found in `shared_buffers`), `read=` (pages that were not there), `temp read=/written=` (spill files)	you want page counts: the chapters' B, hit ratio, cold against warm (Mock v2 C3, A7), whether a hash or sort spilled (C5)

Warning / Gotcha: Things that bite

- `EXPLAIN ANALYZE` *executes*. On a `DELETE` or `UPDATE` it really deletes: wrap it, `BEGIN; EXPLAIN ANALYZE DELETE ...; ROLLBACK;`. On the 13-minute query it takes 13 minutes: use plain `EXPLAIN` first.
- The options go in parentheses, comma-separated: `EXPLAIN (ANALYZE, BUFFERS)`. `EXPLAIN ANALYZE` without parentheses is the old one-option syntax; `EXPLAIN BUFFERS` alone is not valid, BUFFERS needs ANALYZE.
- `cost` is in *units*, `actual time` is in *milliseconds*. They are not comparable to each other; a cost of 9,435 is not 9 seconds.
- `rows` and `actual time` on an inner node of a Nested Loop are *per loop*; multiply by `loops` for totals.
- `read` means "not in `shared_buffers`", not "from the disk": the page may have come from the OS cache (Mock v2 A7, answer b).

Other options you may see: `VERBOSE` prints the output columns of each node; `COSTS off` hides the estimates (used in textbooks for readability); `TIMING off` with ANALYZE skips per-node clocks (cheaper on huge plans); `FORMAT JSON` for tools. `SET max_parallel_workers_per_gather = 0;` removes `Gather` and `Parallel Seq Scan` nodes so the plan looks like the chapters' (the mocks were produced with parallelism off).

7 Anatomy of one node line

```
Seq Scan on orders o  (cost=0.00..9435.00 rows=43388 width=24) (actual time
    =0.012..48.9 rows=42299 loops=1)
     Filter: (order_date >= '2025-12-01'::date)
     Rows Removed by Filter: 457701
     Buffers: shared hit=3185
```

Piece	Meaning
`Seq Scan on orders o`	the operator (the algorithm) and the table it reads; `o` is the alias from the query
`cost=0.00..9435.00`	**estimate**, in cost units, not ms: *startup* cost (before the first row can come out) .. *total* cost. Units: one sequential page = 1, one random page = 4, one row processed = 0.01, one operator on a row = 0.0025. Here: 3,185 pages +500,000 × 0.01 (rows) +500,000 × 0.0025 (the filter) = 9,435
`rows=43388`	**estimated** output rows, from the statistics (Part III), *per loop*
`width=24`	estimated bytes per output row
`actual time=0.012..48.9`	measured ms to the first row .. to the last row (only with ANALYZE)
`rows=42299` (actual)	measured output rows per loop. **Compare with the estimate**: the ratio is the q-error, $\max(\text{est}/\text{act}, \text{act}/\text{est})$; here 1.03, excellent; 10× is a problem, 1,000× is a disaster
`loops=1`	how many times the node ran; the inner side of a Nested Loop runs once per outer row, and then `rows` is *per run*: total = rows × loops
`Filter:`	a predicate tested on every row the node produces; the rows that fail are counted in `Rows Removed by Filter`
`Buffers: shared hit=h read=r`	pages served from `shared_buffers` (h) and pages that were not there and had to be requested from the OS (r); $h + r$ is the page count of the chapters; hit ratio = $h/(h + r)$

8 The tree: who feeds whom

Mechanism: Indentation is the tree; read it bottom-up

The first line is the root, the node that hands rows to the client. Each `->` starts a child, indented under its parent. A node with two children is a join: the **first (upper) child is the outer** side, the **second (lower) child is the inner** side. Rows flow *up*: the deepest leaf runs first. So: find the leaves (scans), walk up each `->`, and at every node ask "what does it do with the rows coming from below?".

```
HashAggregate                          <- root: groups the joined rows, emits 5 totals
  -> Hash Join                         <- joins outer (orders) with inner (the hash of
     customers)
       Hash Cond: (o.cust_id = c.id)
       -> Seq Scan on orders o         <- OUTER = probe side: streamed through the
    hash table
       -> Hash                         <- INNER = build side: read completely first
            -> Seq Scan on customers c
```

Key insight. Two numbers per node matter in every exam question: estimated `rows` against actual `rows`, and `hit` against `read`. Everything else is read only when a question points at it.

Concept: Which nodes block

A node is **blocking** (a pipeline breaker) if it must see all of its input before its first output row: `Sort`, `Hash` (the build side), `HashAggregate`. Everything else **streams**: scans, `Hash Join` (probe side), `Nested Loop`, `Merge Join`, `GroupAggregate`, `Limit`. In `actual time=a..b` a breaker shows $a \approx b$ (nothing comes out until the end); a streaming node shows a wide gap.

9 The nodes you will meet

Node	What it does	Lines under it, and what they tell you
`Seq Scan on t`	reads every page of `t` in order; streaming	`Filter`, `Rows Removed by Filter`: a predicate not served by any index
`Index Scan using i on t`	descends the B-tree `i`, then fetches each matching heap page (random reads); streaming	`Index Cond`: the predicate the index answered. `Index Scan Backward`: walking the leaves in reverse for `ORDER BY ... DESC`
`Index Only Scan`	like Index Scan but the index holds every column needed, so no heap page is read	`Heap Fetches: 0` is the proof; a large number means the visibility map is stale (run `VACUUM`)
`Bitmap Index Scan` + `Bitmap Heap Scan`	collect all matching row ids into a bitmap, sort by page, then read each heap page once in physical order (no random penalty, no re-reads)	`Recheck Cond`; `Heap Blocks: exact=n` is the number of heap pages touched: compare it with B
`Nested Loop`	for each outer row, run the inner child	inner child shows `loops=`$\lvert R\rvert$; total inner rows = rows $\times$ loops
`Hash`	the build side: reads its child completely into a hash table (breaker)	`Buckets`, `Batches` (1 = fit in `work_mem`; > 1 = Grace, spilled to temp files), `Memory Usage`
`Hash Join`	probes the hash with each outer row; streams	`Hash Cond: (a = b)`
`Merge Join`	walks two sorted inputs together	`Merge Cond`; children are `Index Scan`s (already sorted) or `Sort`s
`Sort`	sorts its whole input (breaker)	`Sort Key`; `Sort Method: quicksort Memory: nkB` (fit) or `external merge Disk: nkB` (spilled) or `top-N heapsort` (with LIMIT)
`HashAggregate`	one hash entry per group; emits at the end (breaker)	`Group Key`; `Batches` if it spills
`GroupAggregate`	input arrives sorted on the key; emits a group as soon as the key changes (streaming)	usually sits on a `Sort` or an `Index Scan`
`Aggregate`	a single total (`count(*)`, no GROUP BY)	—
`Limit`, `Materialize`	stop after n rows; keep a child's output in memory to re-scan it	—

10 The five-move method

Technique: Any plan, any question

1. **Find the leaves** (the scans) and name the tables; note the access path (Seq / Index / Bitmap) and the `Filter` or `Index Cond`.
2. **Walk up**: at each node say in one clause what it does with its children's rows; mark the breakers.
3. **Estimate against actual, from the leaves up.** Compute the q-error at each node. The first big gap, lowest in the tree, is the root cause; everything above it inherits the error (a join multiplies it).
4. **Read the sub-lines that cost I/O**: `Buffers` (hit against read, temp read/written), `Sort Method`, `Batches`, `Heap Blocks`, `loops`.
5. **Decide**: is the plan shape right for the real sizes? (nested loop with a big outer, hash with `Batches` $\gg 1$, index scan over most of a table, sort spilling). Then name the fix: `ANALYZE`, `CREATE STATISTICS`, an index, `work_mem`, or "nothing, the plan is right".

11 Worked plan 1: Lab 4 part 1

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.name, SUM(o.amount) AS total
FROM customers c JOIN orders o ON o.cust_id = c.id
WHERE c.country = 'MA' GROUP BY c.name ORDER BY c.name;
```

```
Sort  (cost=1031..1036 rows=1996 width=40) (actual time=121.84..122.07 rows=1996
    loops=1)
  Sort Key: c.name       Sort Method: quicksort  Memory: 205kB       Buffers: shared
    hit=3355
  ->  HashAggregate  (actual time=120.93..121.31 rows=1996 loops=1)   Batches: 1
        ->  Hash Join  (actual time=4.02..105.66 rows=50112 loops=1)
              Hash Cond: (o.cust_id = c.id)
              ->  Seq Scan on orders o  (actual time=0.01..37.95 rows=500000 loops=1)
              ->  Hash  (actual rows=2000 loops=1)  Buckets: 2048  Batches: 1  Memory
     Usage: 117kB
                    ->  Seq Scan on customers c  (rows=2000)  Filter: (country = 'MA
    ')
                          Rows Removed by Filter: 18000
Execution Time: 122.38 ms
```

Moves 1–2. Leaves: Seq Scan of `customers` with the country filter (2,000 of 20,000 kept) feeds `Hash` (build, blocking); Seq Scan of `orders` (500,000 rows, no filter) is the probe side. The Hash Join streams: its first row appears at 4.02 ms, right after the hash is built. HashAggregate (blocking: 120.93..121.31, nothing before the end) collapses 50,112 joined rows into 1,996 names; Sort (blocking) orders them. **Move 3**: estimates match (2,000 est. for MA, 1,996 groups). **Move 4**: `Buffers: shared hit=3355` $= 3{,}185 + 170$ pages, all from the pool; `Batches: 1`, `quicksort Memory: 205kB`: nothing spilled. **Move 5**: right plan; the breakers are Hash, HashAggregate, Sort. The join took $105 - 38 \approx 67$ ms of its own time (its b minus its children's b).

12 Worked plan 2: Mock v2 Part C, the December report (26 pts)

```
-- run 1 (just after a restart)
Sort (cost=10064.00..10064.01 rows=5) (actual time=82.857..82.860 rows=5 loops=1)
  Sort Key: (sum(o.amount)) DESC      Sort Method: quicksort Memory: 25kB
  Buffers: shared hit=3 read=3258
  -> HashAggregate (cost=10063.88..10063.94 rows=5) (actual rows=5 loops=1)   Group
     Key: c.country
       -> Hash Join (cost=298.00..9846.94 rows=43388) (actual rows=42299 loops=1)
             Hash Cond: (o.cust_id = c.id)
             -> Seq Scan on orders o (cost=0.00..9435.00 rows=43388) (actual rows
     =42299 loops=1)
                   Filter: (order_date >= '2025-12-01'::date)    Rows Removed by
     Filter: 457701
                   Buffers: shared read=3185
             -> Hash (actual rows=10000 loops=1)   Buckets: 16384 Batches: 1 Memory
     Usage: 519kB
                   -> Seq Scan on customers c (cost=0.00..173.00 rows=10000) (actual
     rows=10000 loops=1)
                         Buffers: shared read=73
Execution Time: 82.963 ms
-- run 2: same plan, every "read" becomes "hit"      Buffers: shared hit=3258
     Execution Time: 62.288 ms
-- run 3 (work_mem = 64kB): same plan, except
             -> Hash (actual rows=10000 loops=1)   Buckets: 4096 Batches: 8 Memory
     Usage: 80kB
                   Buffers: shared hit=73, temp written=28
  Buffers: shared hit=3258, temp read=183 written=183       Execution Time: 71.524 ms
```

Worked example: The five answers, in the rubric's shape

C1, tree (5 pts). Sort [blocking: all 5 groups before ordering] ← HashAggregate [blocking: a country's total is final only after the last row] ← Hash Join [streaming: once the hash exists, each order is probed and passed up] with outer Seq Scan on orders [streaming; filter per row, 42,299 kept of 500,000] and inner Hash [blocking: the build side, all 10,000 customers first] ← Seq Scan on customers [streaming]. Two of five stream; the first result row appears only at the end.

C2, estimate (4 pts). Estimated 43,388, actual 42,299: q-error = 43,388/42,299 = 1.03, excellent. It comes from the `histogram_bounds` of `order_date` in `pg_stats` (365 distinct days; the fraction of buckets at or above 2025-12-01, about 31/365 = 8.5%, times `reltuples` 500,000).

C3, Buffers (6 pts). 3,258 = 3,185 pages of `orders` +73 of `customers`: the whole of both tables, once. In run 1 they were not in `shared_buffers` (the restart emptied it), so they are `read`; run 1 left them in the pool, so run 2 finds all 3,258 as `hit`. Run 1 is only 82.9/62.3 = 1.3× slower, not 1,000×, because `read` means "not in shared_buffers", *not* "from disk": the OS page cache survived the PostgreSQL restart and served most pages at memory speed; the remainder is a sequential read of 25 MB, ≈ 50 ms at 500 MB/s, not 3,258 random reads; and the CPU work (filtering 500,000 rows, hashing 10,000, probing 42,299) is the same in both runs.

C4, the index (6 pts). The query keeps 42,299/500,000 = 8.5% of the rows, and because the correlation is −0.0015 ≈ 0 those rows are spread

uniformly over the file: about 13 per page, and the probability that a page has none is $(1 - 0.085)^{157} \approx 10^{-6}$. An index scan would therefore visit all 3,185 heap pages anyway, in random order: $3{,}185 \times 4 = 12{,}740$ cost units against 3,185 for the sequential scan, plus about 120 leaf pages. A bitmap scan still reads all 3,185 pages. **No**, the planner keeps the Seq Scan and the index costs space and slows every insert. It would help if the table were physically ordered by date (correlation near 1, after `CLUSTER`, or append-only): then 8.5% of rows is about 271 pages, and a BRIN index does it for almost nothing.

C5, run 3 (5 pts). The build side needed 519 kB; with `work_mem = 64kB` it no longer fits, so the Hash node switched to a Grace hash join: `Batches: 8` means both inputs were partitioned 8 ways, the first batch kept in memory (80 kB) and the other 7 written to temporary files: `temp written=28` on the Hash node is the spilled build partitions (about 4 pages each), and `temp read=183 written=183` at the top is the `orders` rows of batches 2–8, written once and read back once. Same plan shape, memory moved to disk, +15% (tiny files, served by the OS cache). Recommendation: `work_mem` above the build's need with margin, at least 1 MB here, so the default 4 MB is right; set it per statement with `SET LOCAL work_mem = '...'` inside the report's transaction, never globally, because `work_mem` is allocated per sort or hash per connection.

13 Worked plan 3: Mock 1, 3.1, the 13-minute query

```
HashAggregate (actual time=781204 ms)
  -> Nested Loop (cost=.. rows=50) (actual rows=1240000)
       -> Seq Scan on users u (rows=40) (actual rows=200000)     Filter: (city = '
    Casablanca')
       -> Seq Scan on events e (per outer row)   Filter: (user_id = u.user_id AND
    event_type = 'purchase')
```

Move 3 finds it. The lowest wrong node is the leaf `Seq Scan on users`: 40 estimated, 200,000 actual, q-error 5,000. The Nested Loop above is off by $1{,}240{,}000/50 \approx 24{,}800$, but that is the leaf's error multiplied by the join; the rubric wants the leaf. **Move 5**: believing 40 outer rows, the planner chose a nested loop whose inner is a *sequential scan of events, 1,000,000 pages, once per outer row*: $200{,}000 \times 10^6$ page reads. **Causes**: (1) stale statistics (`ANALYZE` last ran at 5M of 50M rows; the `city` distribution is old or absent, so the planner uses a near-default selectivity instead of 10%); (2) correlation and skew it cannot see: `city` determines `country`, and `event_type` is 90/9.5/0.5. **Fix**, in order: `ANALYZE users; ANALYZE events; CREATE STATISTICS s_geo (dependencies) ON country, city FROM users; ANALYZE users; ALTER TABLE events ALTER COLUMN event_type SET STATISTICS 1000;` then `EXPLAIN (ANALYZE, BUFFERS)` again. With 200,000 outer rows and no index the right join is a **hash join** building on the filtered `users` (fits in 8,000 pages of `work_mem`? 200,000 rows $\approx$ 3,000 pages: yes), probing

`events` once: ≈ 1,030,000 pages instead of 2×10^{11}. With `CREATE INDEX ON events(user_id)` an index nested loop becomes possible but 200,000 descents is still worse than one scan.

14 Practice plans (cover the solutions)

All on `adm_lab` (`orders` 500,000 rows / 3,185 pages / 157 rows per page, `customers` 10,000 / 73; `amount` uniform 5..105; a B-tree on every column mentioned). Numbers are realistic, not measured.

Practice: P1: a point lookup

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE id = 4275;
Index Scan using orders_pkey on orders  (cost=0.42..8.44 rows=1 width=24) (actual
      time=0.030..0.031 rows=1 loops=1)
  Index Cond: (id = 4275)
  Buffers: shared hit=4
```

(a) How many page reads, and what are they? (b) Why `Index Cond` and not `Filter`? (c) Why is the cost about 8 and not 3,185? (d) What would the same query print with no index?

Solution

(a) 4 pages: the B-tree on 500,000 ids has 3 levels (fanout ≈ 300: 500,000 → 1,667 → 6 → 1), root + inner + leaf, then 1 heap page: levels +1. (b) The index answered the predicate, so no row was fetched and rejected; a `Filter` line would mean rows were read and then tested. (c) The planner charges the random pages it expects: about two random reads (2 × 4) plus CPU; the 0.42 startup is the descent's CPU. (d) `Seq Scan on orders ... Filter: (id = 4275) Rows Removed by Filter: 499999 Buffers: shared hit=3185`, cost 9,435: every page, every row tested.

Practice: P2: a nested loop with an index, and `loops`

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM customers c JOIN orders o ON o.cust_id = c.id WHERE c.id < 10;
Nested Loop  (cost=0.71..1894.00 rows=450 width=60) (actual time=0.040..0.520
    rows=432 loops=1)
  Buffers: shared hit=471
  ->  Index Scan using customers_pkey on customers c  (cost=0.29..8.45 rows=9
     width=36)
        (actual time=0.015..0.020 rows=9 loops=1)
        Index Cond: (id < 10)
  ->  Index Scan using orders_cust_id_idx on orders o  (cost=0.42..209.00 rows=50
     width=24)
        (actual time=0.010..0.045 rows=48 loops=9)
        Index Cond: (cust_id = c.id)
Execution Time: 0.6 ms
```

(a) Which side is outer, which inner, and how many times did the inner run? (b) The inner says `rows=48`; how many orders were joined in total?

(c) Why is this the right plan here, and what plan does the same query get without the `WHERE` (Lab 4.5)? (d) Account for the 471 buffers.

Solution

(a) Outer = the first child, 9 customers; inner = the index scan on `orders`, run `loops=9` times, once per outer row. (b) $48 \times 9 = 432$, the root's actual rows; `rows` on an inner node is per loop. (c) A tiny outer with an index on the inner key: 9 descents of 3 pages plus about 50 heap pages each, ≈ 470 pages, in 0.6 ms; without the filter the outer is 10,000 rows, 10,000 descents (`loops=10000`), and the planner switches to a Hash Join reading each table once (3,185+73 pages, ≈ 230 ms); forced to nested loop it takes ≈ 640 ms. (d) $9 \times (3 \text{ index} + 48 \text{ heap}) \approx 459$, plus the 3-page descent and leaf for `customers` and their heap pages: ≈ 471.

Practice: P3: a bitmap scan that touches every page

```
EXPLAIN (ANALYZE, BUFFERS) SELECT count(*) FROM orders WHERE amount < 10;
Aggregate  (cost=5131.00..5131.01 rows=1 width=8) (actual time=45.1..45.1 rows=1
    loops=1)
  Buffers: shared hit=3254
  ->  Bitmap Heap Scan on orders  (cost=468.00..5068.50 rows=25000 width=0) (
     actual time=3.2..41.0 rows=24871 loops=1)
        Recheck Cond: (amount < 10)
        Heap Blocks: exact=3185
        ->  Bitmap Index Scan on orders_amount_idx  (cost=0.00..461.75 rows=25000
      width=0)
              (actual time=2.9..2.9 rows=24871 loops=1)
              Index Cond: (amount < 10)
```

(a) Where does the estimate 25,000 come from? (b) `Heap Blocks: exact=3185`: what does it say, and why that number? (c) Why a bitmap scan rather than a plain Index Scan? (d) Is it really better than a Seq Scan here? (e) What would `amount < 6` print?

Solution

(a) `amount` is uniform on 5..105, so the histogram puts 5% of the rows below 10: $0.05 \times 500{,}000 = 25{,}000$. (b) The heap scan touched 3,185 distinct pages, i.e. *every page* of `orders`: $0.05 \times 157 \approx 8$ matches per page, and $(0.95)^{157} \approx 3 \times 10^{-4}$ is the chance a page has none. (c) 25,000 random fetches would cost $25{,}000 \times 4 = 100{,}000$; the bitmap sorts the row ids by page and reads each page once, in order, for $\approx 3{,}185$. (d) Barely: it reads the same 3,185 pages (+69 index leaf pages = 3,254 buffers), its gain is CPU only (25,000 rows checked instead of 500,000: cost 5,131 against the Seq Scan's 9,435). (e) 1%: 5,000 rows, ≈ 0.8 per page, $(0.99)^{157} = 21\%$ of pages empty: `Heap Blocks` $\approx 2{,}500$; still a bitmap scan; a plain Index Scan needs $s < 1/(4 \cdot 157) = 0.16\%$.

Practice: P4: a forced merge join with a spilled sort (Lab 4.2)

```
SET enable_hashjoin = off;
EXPLAIN ANALYZE SELECT * FROM customers c JOIN orders o ON o.cust_id = c.id;
Merge Join  (cost=66930..75180 rows=500000 width=60) (actual time=247.3..432.1
    rows=500000 loops=1)
  Merge Cond: (c.id = o.cust_id)
  ->  Index Scan using customers_pkey on customers c  (actual time=0.02..6.1 rows
    =10000 loops=1)
  ->  Sort  (cost=66929..68179 rows=500000 width=24) (actual time=247.0..310.4
    rows=500000 loops=1)
        Sort Key: o.cust_id
        Sort Method: external merge  Disk: 8184kB
        ->  Seq Scan on orders o  (actual time=0.01..38.2 rows=500000 loops=1)
Execution Time: 432.1 ms
```

(a) Why does the first row of the join appear only at 247 ms when the hash join's appeared at 4 ms? (b) Why is `customers` read with an Index Scan and no Sort? (c) What does the `Sort Method` line say, and what setting would change it, to what? (d) Why did the planner not pick this plan by itself?

Solution

(a) Merge join needs both inputs sorted; the Sort under it is a breaker that must read and sort all 500,000 orders (247 ms) before emitting its first row; the hash join only had to build a 10,000-row hash. (b) The primary-key index delivers `customers` already in `id` order, so the sorted input is free. (c) The sort did not fit in `work_mem` (4 MB): it ran an external merge sort with 8 MB of temporary files. `SET work_mem = '32MB'` (anything above the run size) turns it into `Sort Method: quicksort Memory: 16000kB`. (d) Hash join reads each input once with no sort ($\approx$ 230 ms); merge pays the 500,000-row external sort; the planner costs both and hash is cheaper. `RESET ALL;` afterwards.

Practice: P5: an index-only scan

```
EXPLAIN (ANALYZE, BUFFERS) SELECT cust_id FROM orders WHERE cust_id = 42;
Index Only Scan using orders_cust_id_idx on orders  (cost=0.42..6.30 rows=50
    width=4)
     (actual time=0.02..0.04 rows=48 loops=1)
  Index Cond: (cust_id = 42)
  Heap Fetches: 0
  Buffers: shared hit=4
```

(a) Why only 4 buffers for 48 rows, when P2's inner side needed about 51 per customer? (b) Where does `rows=50` come from? (c) When would `Heap Fetches` be 48, and what fixes it?

Solution

(a) The query needs only `cust_id`, which the index itself stores: 3 index pages to descend plus the one leaf holding customer 42's 48 entries, and no heap page at all; P2 needed every row's other columns. (b) `reltuples`

/ `n_distinct` = 500,000/10,000 = 50 (the quiz's A1). (c) When the visibility map does not yet mark those heap pages all-visible (after inserts or updates, before a `VACUUM`); each row then needs a heap visit to check visibility. `VACUUM orders;` refreshes the map.

Part III

The PostgreSQL instruments, one per mock question

Each instrument below is a box: what question it answers, the exact command, what the output looks like, how to read it, and the mock question that needs it. "The planner" never looks at the table when it plans; it looks at the numbers these views expose, which is why half of the instruments are about statistics.

15 Page counts: `pg_class`, `pg_relation_size`

Interface / API: How big is a table, in pages? (Mock 1 3.2c; Mock v2 A2, A7; Quiz 5 A3)

```
SELECT relname, reltuples::bigint AS rows_est, relpages FROM pg_class
    WHERE relname = 'orders';
--  orders | 500000 | 3185                    <- what the planner
    believes (refreshed by ANALYZE)
SELECT pg_relation_size('orders') / 8192 AS pages;            -- 3185:
    the real file, now
SELECT pg_size_pretty(pg_relation_size('orders'));            -- 25 MB
     (heap only)
SELECT pg_size_pretty(pg_total_relation_size('orders'));      -- heap +
    indexes + TOAST
SELECT pg_size_pretty(pg_indexes_size('orders'));             -- indexes
     alone (mock 2.3: the price)
```

Reading it. `relpages` is B, the chapters' page count; `reltuples` is N. The Seq Scan cost the plan will print is $B \times 1 + N \times 0.01$ ($+N \times 0.0025$ per filter): $3{,}185 + 5{,}000 = 8{,}185$ with no WHERE, 9,435 with one (Quiz 5 A3). Rows per page = N/B = 157. If `pg_relation_size/8192` and `relpages` differ a lot, the statistics are stale (§4). After deleting half the rows and a plain `VACUUM`, `pg_relation_size` still says 3,185 pages (Mock v2 A2).

16 Looking inside a page: `pageinspect`, `ctid`

Interface / API: How many rows on a page, really? (Mock 1 3.2c; drill D7)

```
CREATE EXTENSION IF NOT EXISTS pageinspect;
SELECT count(*) FROM heap_page_items(get_raw_page('orders', 0));
    -- ~157 (events: ~50)
SELECT lp, lp_off, lp_len, t_xmin, t_xmax
FROM heap_page_items(get_raw_page('orders', 0)) LIMIT 3;
--  1 | 8152 | 40 | 912 | 0        <- slot 1 at offset 8152, 40 bytes,
    created by xid 912, not deleted
--  2 | 8112 | 40 | 912 | 0        <- offsets decrease: tuples grow up
    from the bottom of the page
SELECT ctid, id FROM orders LIMIT 3;                                    --
     (0,1) (0,2) (0,3): (page, slot)
```

Reading it. `get_raw_page(table, n)` fetches page n as bytes; `heap_page_items` decodes its slot directory: one row per slot (`lp`), the tuple's offset and length, the creating and deleting transaction ids. `t_xmax <> 0` means a dead tuple waiting for `VACUUM`. For Mock 1 3.2(c): the two commands are `pg_relation_size('events')/8192` (expect 1,000,000) and `count(*)` over page 0 (expect about 50); the `jsonb` payload does not distort the count because it lives out of line in TOAST.

17 What the planner believes about a column: `pg_stats`

Interface / API: Where do `rows=` estimates come from? (Mock v2 A6, C2, C4; Quiz 5 A1–A2, B1; Lab 5.2–5.3)

```
\x                                          -- vertical display, the rows
    are wide
SELECT attname, n_distinct, null_frac, most_common_vals,
    most_common_freqs,
       histogram_bounds, correlation
FROM pg_stats WHERE tablename = 'orders';
-- attname           | cust_id
-- n_distinct        | 10000             <- 10,000 distinct customers
-- most_common_vals  | NULL              <- no value stands out
-- histogram_bounds  | {1,101,203,...}   <- 100 equal-count buckets
-- correlation       | 0.0012            <- physical order unrelated to
    cust_id
-- attname           | amount
-- n_distinct        | -0.02  (negative = a fraction of the rows: 2% of
    500,000 = 10,000 values)
-- histogram_bounds  | {5.00,6.01,...,104.99}
-- attname           | order_date
-- n_distinct        | 365
-- correlation       | -0.0015
```

Mechanism: The five estimation rules, read straight off `pg_stats`

Predicate	Selectivity s	Example (`orders`, $N = 500{,}000$)
`col = v`, v in `most_common_vals`	its `most_common_freqs` entry	`country = 'MA'`: 0.20 exactly
`col = v`, v not an MCV	$(1 - \sum \text{MCV freqs})/(\text{n_distinct} - \#\text{MCV})$; with no MCVs, 1/n_distinct	`cust_id = 42`: $1/10{,}000 \to 50$ rows (Quiz A1)
`col > v`, `BETWEEN`	fraction of the histogram buckets covered (100 buckets = 1% each)	`amount > 95`: 10% $\to$ 50,000; `BETWEEN 20 AND 40`: 20% (Quiz A2)
`p1 AND p2`	$s_1 \times s_2$ (**independence**, wrong when columns are correlated)	`amount > 95 AND product_id = 7`: $0.1 \times 1/200 \to 250$ rows (Mock v2 A6)
`R JOIN S ON R.k = S.k`	$\lvert R\rvert \cdot \lvert S\rvert / \max(\text{nd}_R, \text{nd}_S)$	2,000 MA customers $\times$ 500,000 orders /10,000 = 100,000

`n_distinct` negative is a fraction of the rows (-1 = unique). `correlation` in $[-1, 1]$: near ± 1 the heap is physically in column order (cheap range index scans, BRIN); near 0 a range's rows are spread over every page (Mock v2 C4). The histogram has 100 buckets by default (`default_statistics_target`); `ALTER TABLE t ALTER COLUMN c SET STATISTICS 1000;` then `ANALYZE` gives finer buckets and a longer MCV list for a skewed column such as `event_type`.

18 Fresh or stale: `ANALYZE`, `pg_stat_user_tables`

Interface / API: Are the statistics current? (Mock 1 3.1; Quiz 5 A8; Lab 5.4)

```
SELECT relname, n_live_tup, n_dead_tup, last_analyze, last_autoanalyze
FROM pg_stat_user_tables WHERE relname = 'orders';
--  orders | 800000 | 0 | 2026-10-01 09:12 | NULL      <- 800,000 live
    rows, analysed when it had 500,000
ANALYZE orders;                 -- re-sample the table, rewrite reltuples
    , relpages, pg_stats (seconds)
VACUUM ANALYZE orders;          -- also reclaim dead tuples and refresh
    the visibility map
```

Reading it. `n_live_tup` is the current row count; `reltuples` in `pg_class` is what the planner uses. When they differ by the growth factor, every estimate on that table is off by that factor: the Lab 5.4 experiment (insert 300,000 rows, `EXPLAIN ANALYZE` shows `rows=75000` against 120,000 actual; `ANALYZE orders` snaps it back). Autovacuum re-analyses after about 10% of a table changes, so a table that grew 10× with autovacuum off (Mock 1: 5M to 50M) is the classic stale case. **First command of any diagnosis: `ANALYZE t`; then re-run the EXPLAIN.**

19 Correlated columns: `CREATE STATISTICS`

Interface / API: Fix an under-estimate on `country AND city` (Mock 1 3.1, 3.5a; Quiz 5 A5; Lab 5.5; drill D12)

```
EXPLAIN SELECT * FROM customers WHERE country = 'MA' AND city = '
    Casablanca';
--  Seq Scan on customers (rows=400)        <- 0.2 x 0.2 x 10,000:
    independence, 5x too low
CREATE STATISTICS s_geo (dependencies) ON country, city FROM customers;
ANALYZE customers;                           -- the new statistics object
     is empty until ANALYZE fills it
EXPLAIN SELECT * FROM customers WHERE country = 'MA' AND city = '
    Casablanca';
--  Seq Scan on customers (rows=2000)       <- the functional dependency
    city -> country is now known
```

Reading it. The planner multiplies selectivities as if columns were independent; when one column determines the other (every Casablanca customer is in MA) the product is too low by exactly the factor of the implied column, sel(country) = 0.2. Mock 1 3.5(a): $0.2 \times 0.1 \times 2{,}000{,}000 = 40{,}000$ estimated against $0.1 \times 2{,}000{,}000 = 200{,}000$ true. The fix is the pair of commands above; `dependencies` is the statistics kind (others: `ndistinct`, `mcv`). A 5× error at a leaf becomes a wrong join algorithm above it (3.1).

20 Steering the planner: `SET enable_*`, `work_mem`, `SET LOCAL`, `RESET`

Interface / API: Force an algorithm or shrink memory, for an experiment (Lab 4.2–4.3; Mock v2 B3d, C5, D2)

```
SHOW work_mem;                      -- 4MB (default): the M of one sort or
    one hash build, per operator
SHOW shared_buffers;                -- 128MB default; the lab server: 4 GB
    = 500,000 pages
SET enable_hashjoin = off;          -- planner now pays 10^10 for a hash
    join: falls back to merge join
SET enable_mergejoin = off;         -- ... and then to nested loop
SET enable_seqscan = off;           -- shows the index plan it rejected,
    with its (higher) cost
SET work_mem = '64kB';              -- make a sort spill / a hash batch:
    Sort Method: external merge
SET work_mem = '256MB';             -- make it fit:
    Sort Method: quicksort
RESET work_mem;   RESET enable_hashjoin;   RESET ALL;    -- back to
    defaults; do it between lab parts
BEGIN;  SET LOCAL work_mem = '64MB';  <the report>;  COMMIT;   -- per
    statement, the production way
```

Reading it. The `enable_*` switches are *penalties*, not prohibitions: if no other plan exists, the penalised one still runs, with absurd cost numbers that distort every estimate near it (Mock v2 D2). `work_mem` is allocated per sort or hash operation per connection: 256 MB × 50 connections × 2 sorts = 25 GB, which is why it is raised with `SET LOCAL` for a known heavy statement and never globally (Mock v2 C5). The proving lines: `Sort Method: external merge Disk: 8184kB` against `quicksort Memory: 71680kB`; `Batches: 8` against `Batches: 1`; `Index Scan ... loops=20000` for the forced nested loop.

21 The buffer pool: `pg_buffercache` and the `Buffers` line

Interface / API: Which tables are in memory right now? (Mock 1 3.6c; drill D9)

```
CREATE EXTENSION IF NOT EXISTS pg_buffercache;
SELECT c.relname, count(*) AS buffers, pg_size_pretty(count(*) * 8192) AS
    size
FROM   pg_buffercache b
JOIN   pg_class c ON b.relfilenode = pg_relation_filenode(c.oid)
WHERE  b.reldatabase IN (0, (SELECT oid FROM pg_database WHERE datname =
    current_database()))
GROUP  BY c.relname ORDER BY buffers DESC LIMIT 10;
--  orders        | 3185 | 25 MB        <- the whole table is cached
    after one scan
```

```
--  orders_pkey    |  412 | 3296 kB
--  customers      |   73 | 584 kB
```

Reading it. One row per relation, how many of the pool's frames hold its pages. Run it before and after a query to see what the query pulled in or evicted. For Mock 1 3.6: a cold scan of 1,000,000 pages through a 500,000-page pool has a hit ratio of about 0 (every page read once, nothing re-touched), and plain LRU would evict everything else; PostgreSQL gives a scan of a table larger than `shared_buffers`/4 a private 256 KB ring buffer (32 pages), so after the scan `events` shows a few dozen buffers, not hundreds of thousands.

Interface / API: How much came from the pool? (Mock v2 A7, C3; Mock 1 2.2)

```
Buffers: shared hit=40 read=3145                <- 3,185 pages requested: 40
    were in shared_buffers,
                                                   3,145 were not and were asked
     from the OS
Buffers: shared hit=3258, temp read=183 written=183   <- temp = spill files of a
    sort or hash (not the pool)
```

Reading it. Hit ratio $h = \text{hit}/(\text{hit} + \text{read}) = 40/3{,}185 = 1.3\%$ for a cold scan; 100% on the second run. `read` is "not in `shared_buffers`": the page may still have come from the OS page cache at memory speed, which is why a cold run is 1.3× slower and not 1,000× (Mock v2 C3, A7 answer b). Effective access time $= h \cdot t_{\text{mem}} + (1-h)\, t_{\text{disk}}$. `hit + read` per node is the chapters' page count: use it to check B (3,185) and to spot a CPU-bound query (all hits, no reads, still slow: Mock 1 2.2).

22 Which indexes are used: `pg_stat_user_indexes`, `pg_indexes`

Interface / API: Which indexes to keep (Mock 1 2.3; drill D11)

```
SELECT indexrelname, idx_scan, pg_size_pretty(pg_relation_size(indexrelid
    )) AS size
FROM   pg_stat_user_indexes WHERE relname = 'events' ORDER BY idx_scan;
--  events_device_idx   |     0 | 1071 MB      <- never used since the
    counters were reset: drop it
--  events_user_id_idx  | 98213 | 1071 MB
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'events';
    -- the CREATE INDEX text
```

Reading it. `idx_scan` counts how many times the planner chose that index since statistics were reset. An index with `idx_scan = 0` after a representative period costs its size in the pool and one random leaf write per insert for nothing: drop it. Before trusting a zero, make sure the period covered the monthly reports.

23 One column out: `COPY` to Parquet and DuckDB

Interface / API: Export `events.ts` and count per day (Mock 1 3.2b; drill D10)

```
-- PostgreSQL side (pg_parquet extension); FORMAT csv works everywhere
    and DuckDB converts it
COPY (SELECT ts FROM events) TO '/data/events_ts.parquet' (FORMAT parquet
    );
-- DuckDB side
SELECT ts::date AS day, count(*) FROM read_parquet('/data/events_ts.
    parquet') GROUP BY day ORDER BY day;
```

Reading it. `COPY (query) TO file` writes a query's result to a file on the server. Why it is cheaper: the PostgreSQL row store must read all 1,000,000 pages (8 GB, 16 s cold) to see one 8-byte column; the Parquet file holds that column alone ($50\text{M} \times 8 = 400$ MB raw, tens of MB delta-encoded) and DuckDB reads it vectorised, skipping row groups by their min/max. The syntax is 1 point; the argument (one column, compression, skipping) is the rest.

Summary: If you remember eight commands

1. `EXPLAIN (ANALYZE, BUFFERS) q;` and read `rows=` against actual, `hit` against `read`.
2. `ANALYZE t;` first, always.
3. `SELECT attname, n_distinct, most_common_vals, histogram_bounds, correlation FROM pg_stats WHERE tablename = 't';`
4. `CREATE STATISTICS s (dependencies) ON a, b FROM t; ANALYZE t;`
5. `SELECT relpages, reltuples FROM pg_class WHERE relname = 't';` and `pg_relation_size('t')/8192`.
6. `SET work_mem = '64kB';` / `SET enable_hashjoin = off;` / `RESET ALL;`
7. `CREATE EXTENSION pg_buffercache;` and the relname/count(*) query.
8. `CREATE INDEX ON t (col) [WHERE ...] [INCLUDE (...)]`, then `EXPLAIN` to prove it is used.

24 The map: mock question to what you type

Mock question	Type / read	Section here
Mock 1 3.1 bad plan	find the leaf with the worst q-error; `ANALYZE` both tables; `CREATE STATISTICS ... (dependencies)`; `SET STATISTICS 1000`; re-`EXPLAIN`; name hash join	II.7, III.4, III.5
Mock 1 3.2 storage	`pg_relation_size/8192`; `heap_page_items(get_raw_page(...))`; `COPY ... (FORMAT parquet)`; `read_parquet`	III.1, III.2, III.9
Mock 1 3.3 indexes	six `CREATE INDEX` forms; (d) none, $0.9 \gg 1/(4 \cdot 50)$	I.4
Mock 1 3.5 estimation	independence product; true count; `CREATE STATISTICS`; view 0.9 against purchase 0.005	III.3, III.5
Mock 1 3.6 buffers	hit ratio ≈ 0; ring buffer; the `pg_buffercache` query	III.7
Mock 1 3.7 SQL	the 30-day group-by; the one-draw skewed `generate_series` insert; `ANALYZE`	I.2, I.3
Mock v2 A6, Quiz 5 A1–A3	selectivity from `pg_stats`; Seq Scan cost $B + 0.01N (+0.0025N)$	III.1, III.3
Mock v2 A7, C3	`hit/read`: "not in shared_buffers", OS cache, 1.3×	III.7
Mock v2 C1, C2, C4, C5	tree and breakers; q-error; correlation and $(1-s)^r$; `Batches: 8` and temp lines; `SET LOCAL work_mem`	II.6, III.3, III.6
Lab 4	`\timing`; `SET enable_*`; `work_mem` 64kB / 256MB; `Sort Method`; `RESET ALL`	III.6
Lab 5	`rows=` against actual per node; `pg_stats`; `FILTER` selectivity; stale insert then `ANALYZE`; `CREATE STATISTICS`	II.2, III.3–III.5